CS7IS2: Artificial Intelligence Assignment 3

Faith Olopade - 21364066

Introduction **2**
Methodology **2**
Game Implementations 2
Default Opponent 3
Algorithm Implementations 3
Minimax (No Pruning) 3
Minimax with Alpha Beta Pruning 3
Tabular Q Learning 3
Deep Q Network (DQN) 3
Evaluation Metrics 3
Connect 4 Scalability Analysis **4**
Tic Tac Toe: Algorithms vs Default Opponent **4**
Analysis 5
Training Convergence 6
Decision Time 7
Connect 4: Algorithms vs Default Opponent **7**
Analysis 8
Training Convergence 9
Decision Time 9
Head to Head Matchups **10**
Tic Tac Toe Head to Head 10
Connect 4 Head to Head 12
Overall Comparison **13**
Statistical Analysis **15**
Tic Tac Toe Multi Run Analysis 15
Connect 4 Multi Run Analysis 16
Hyperparameter Sensitivity Analysis **17**
Connect 4: Search Depth Variation 17
TTT: Q Learning Training Episode Variation 18
TTT: Q Learning Learning Rate Variation 19
Limitations **20**
Conclusion **20**
References **20**
Appendix 1: Minimax for Tic Tac Toe **21**
Appendix 2: Minimax for Connect 4 **25**
Appendix 3: Q Learning for Tic Tac Toe **29**
Appendix 4: Q Learning for Connect 4 **34**
Appendix 5: DQN for Tic Tac Toe **34**
Appendix 6: DQN for Connect 4 **43**

Introduction

This report presents a comparative analysis of adversarial search and reinforcement learning algorithms applied to two board games Tic Tac Toe (3x3) and Connect 4 (6x7). Four algorithms were implemented and evaluated: Minimax without pruning, Minimax with Alpha Beta pruning, tabular Q Learning and a Deep Q Network (DQN). Each algorithm was tested against a semi intelligent default opponent and against each other, with performance measured across multiple metrics including win rate, draw rate, decision time, training convergence and statistical consistency across independent runs.

The key research questions addressed are:

1. how do search based methods compare to learning based methods in terms of playing strength,
2. how does game complexity affect the relative performance of each approach
3. What are the trade-offs between computational cost at decision time versus upfront training cost and (4) how sensitive are the algorithms to their hyperparameters?

Methodology

Game Implementations

Tic Tac Toe uses a 3x3 grid with 9 possible positions and a maximum game length of 9 moves. Connect 4 uses a standard 6x7 board where pieces drop to the lowest available row in a column, with a maximum of 42 moves. Both games use player identifiers of +1 and -1, with the board stored as a NumPy array. Both games expose a consistent interface (reset, get_valid_actions, make_move, check_winner) enabling all algorithms to operate on either game without modification.

Default Opponent

The default opponent follows a three priority decision hierarchy as specified in the assignment brief:

1. if a winning move exists, take it immediately
2. if the opponent has a winning move, block it
3. otherwise, select a random valid action.

This produces a semi intelligent opponent that is significantly stronger than random play but does not perform multi move lookahead.

Algorithm Implementations

Minimax (No Pruning)

The standard Minimax algorithm recursively evaluates all possible game states by alternating between maximising and minimising players. A transposition table is used to cache previously evaluated states and avoid redundant computation. For Tic Tac Toe, full depth search is feasible. For Connect 4, a depth limit of 4 is applied with a heuristic evaluation function that scores board positions based on connected piece counts in horizontal, vertical and diagonal windows of length 4, with a centre column bonus.

Minimax with Alpha Beta Pruning

Alpha Beta pruning maintains two bounds (alpha and beta) during the Minimax search, pruning branches that cannot affect the final decision. This produces identical results to standard Minimax but visits significantly fewer nodes. The same depth limit (4) and evaluation function are used for Connect 4.

Tabular Q Learning

Q Learning is an off policy reinforcement learning algorithm that learns state action values (Q values) through temporal difference updates. The agent uses epsilon greedy exploration during training with epsilon decaying from 1.0 to 0.01. The Q table maps (state, action) pairs to learned values. For Tic Tac Toe, the agent was trained for 80,000 episodes against the default opponent. For Connect 4, it was trained for 30,000 episodes against a random opponent, as the brief specifies this approach for the larger state space. Key hyperparameters: learning rate 0.1, discount factor 0.95.

Deep Q Network (DQN)

The DQN uses a feedforward neural network implemented entirely in NumPy (no external deep learning frameworks). The architecture consists of an input layer (board size), two hidden layers with ReLU activations (64/32 for TTT, 128/64 for C4) and a linear output layer producing Q values for each action. Stability features include Huber loss to prevent gradient explosion, Adam optimiser with adaptive learning rates, experience replay with a buffer of 30,000 transitions and a target network updated via soft updates (tau=0.05). Training was 10,000 episodes for both games.

Evaluation Metrics

Performance is evaluated using win rate (percentage of games won), draw rate, loss rate, average decision time per game (measuring computational efficiency), training convergence (rolling 500 episode win rate and reward for RL agents), nodes visited (for search algorithms),

multi run statistical consistency (3 independent runs with mean and standard deviation for RL agents on both games) and hyperparameter sensitivity analysis (varying depth, learning rate and training episodes). These metrics provide a multi dimensional view of algorithm performance as recommended in the marking guidelines.

Connect 4 Scalability Analysis

Before running Connect 4 experiments, a scalability test was conducted to confirm that full depth Minimax is infeasible. Both algorithms were run from an empty board with increasing depth limits until a 30 second budget was exceeded.

Algorithm	Max Depth	Total Nodes	Time (s)	Max Possible
Minimax (no pruning)	8	740,187	90.6	42
Alpha-Beta pruning	8	202,404	48.4	42

Table 1: Scalability test results from an empty Connect 4 board

Both algorithms reached only depth 8 out of a maximum 42 moves. Minimax without pruning visited 740,187 nodes in 90.6 seconds, while Alpha Beta reduced this to 202,404 nodes (a 3.66x reduction). The exponential branching factor (approximately 7 at each level) makes full depth search completely infeasible extrapolating from the observed growth rate, a full 42 move search would require visiting on the order of 10^{25} nodes.

Based on these findings, depth limited search with depth=4 was adopted for all Connect 4 Minimax experiments. A heuristic evaluation function was implemented that scores partial board positions by counting pieces in connected windows across all four directions, weighting near complete sequences heavily (4 in a row: +1000, 3 with empty: +50, 2 with 2 empty: +5) with equivalent penalties for opponent sequences. A centre column bonus of +3 per piece encourages strategically important centre control.

Tic Tac Toe: Algorithms vs Default Opponent

Algorithm	Win %	Draw %	Loss %	Avg Time	Games
Minimax	53.0%	47.0%	0.0%	67.8ms	100
Alpha-Beta	59.0%	41.0%	0.0%	126.4ms	100

Q-Learning	42.0%	13.0%	45.0%	0.0ms	100
DQN	17.0%	52.0%	31.0%	0.1ms	100

Table 2: Tic Tac Toe performance against the default opponent (100 games each).

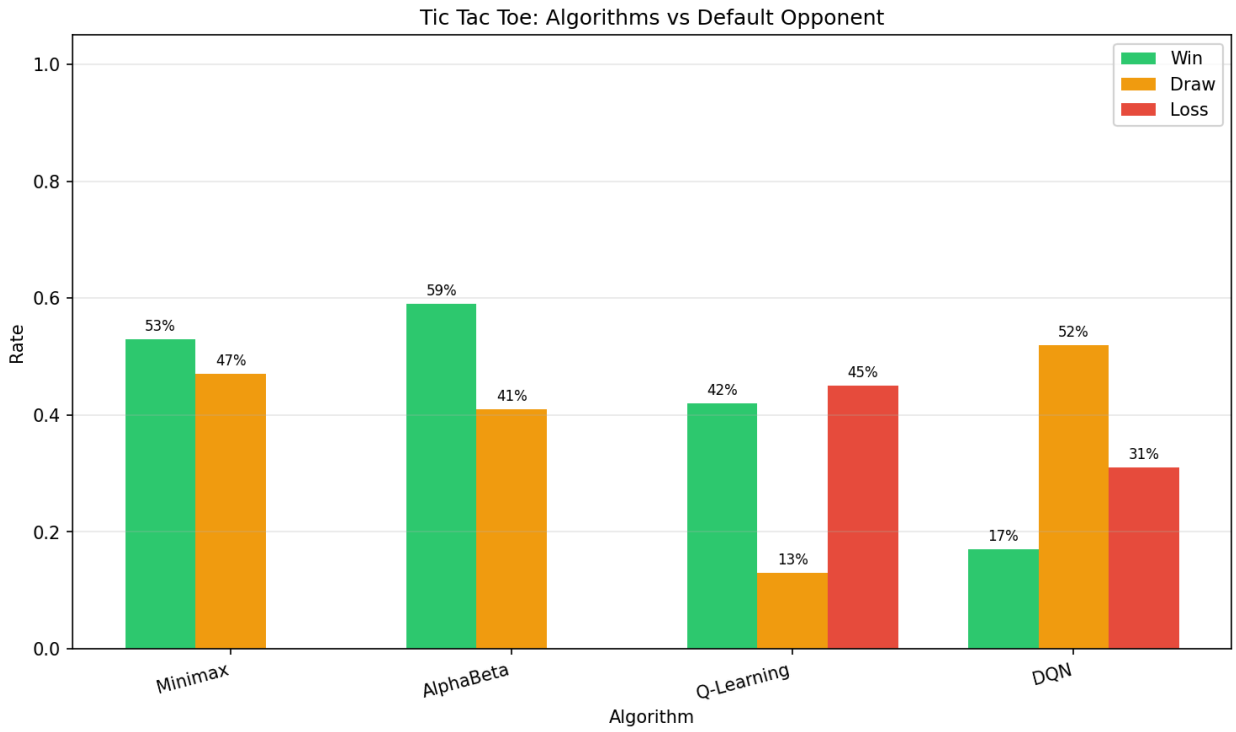

Figure 1: Win, draw, and loss rates for each algorithm against the default opponent in Tic Tac Toe.

Analysis

Both Minimax variants achieved 0% loss rates, confirming that full depth search produces optimal play in Tic Tac Toe. The win/draw split (53-59% wins, 41-47% draws) is determined entirely by who moves first when the optimal agent goes first, it can force a win, when it goes second against a semi intelligent opponent, optimal play typically results in a draw.

Q Learning achieved a 42% win rate but suffered a 45% loss rate, indicating that 80,000 training episodes against the default opponent produced a competent but imperfect policy. The agent learned to exploit some of the default opponent's weaknesses but still makes suboptimal moves in certain board states. The low draw rate (13%) suggests Q Learning plays aggressively, leading to decisive outcomes.

DQN achieved 17% wins with the highest draw rate (52%) and a 31% loss rate. This indicates DQN learned a more conservative, defensive strategy compared to Q Learning; it avoids losing positions more effectively but fails to exploit winning opportunities as aggressively. This pattern

is characteristic of function approximation approaches that learn smoother value estimates rather than the sharp state specific values that tabular methods capture.

Training Convergence

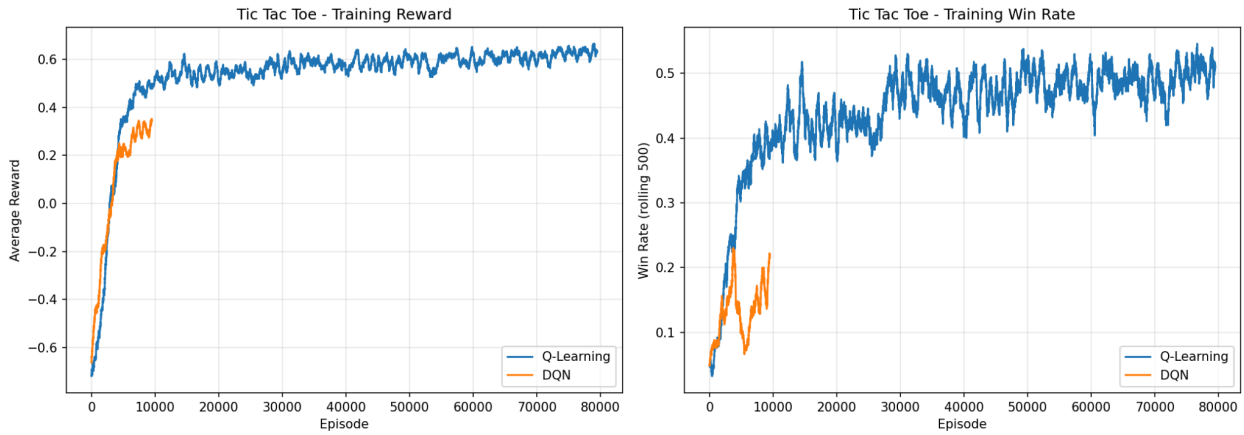

Figure 2: Training reward and win rate curves for RL agents on Tic Tac Toe (rolling 500-episode average).

Q Learning shows steady improvement over 80,000 episodes, with the rolling win rate climbing from near zero to a plateau around 45-50%. The reward curve converges to approximately 0.6, indicating consistent positive outcomes. DQN's shorter training window (10,000 episodes) shows early improvement peaking around 20% win rate before plateauing at 15-22%. The Huber loss remained stable and low (0.006-0.03), confirming training stability even if the policy did not converge to as strong a solution as Q Learning.

Decision Time

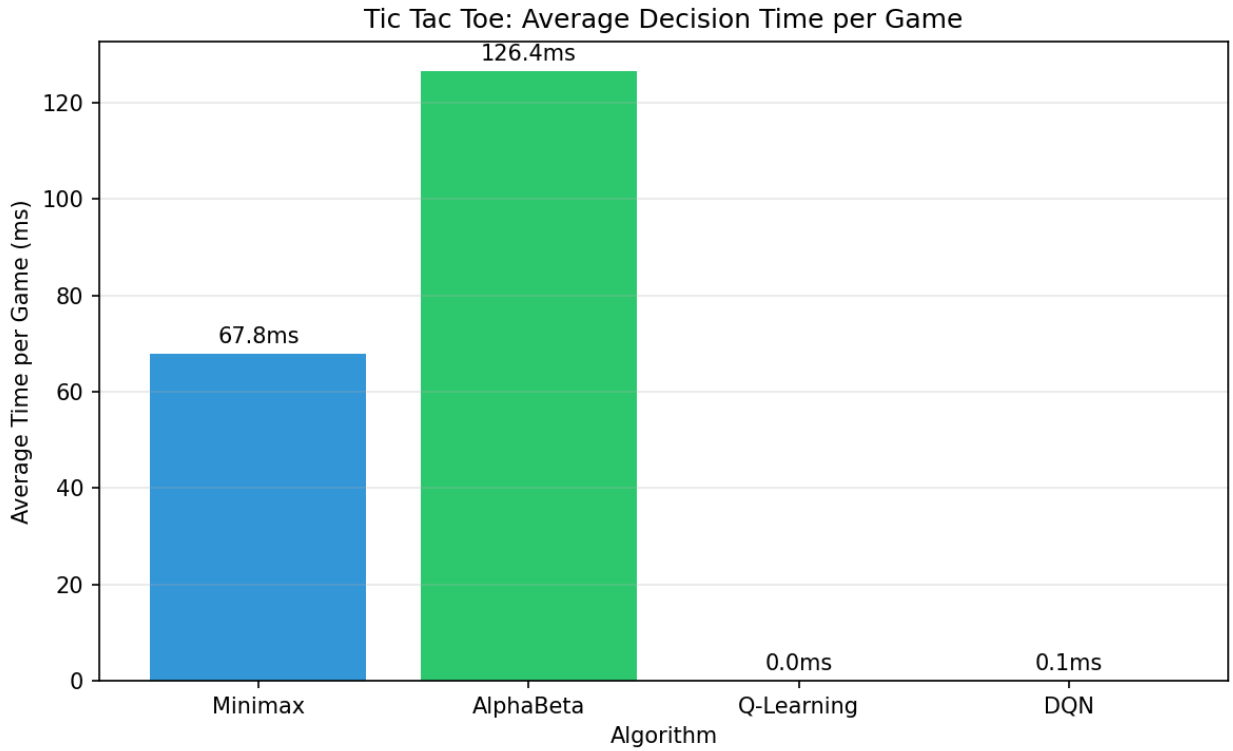

Figure 3: Average decision time per game for Tic Tac Toe.

A striking trade off emerges between playing strength and decision speed. Minimax requires 67.8ms per game and Alpha Beta 126.4ms. Both RL agents decide in under 0.1ms since they perform only a table lookup (Q Learning) or a single forward pass through a small neural network (DQN). This represents a 600-1200x speed advantage for RL at decision time, though this ignores the upfront training cost of thousands of episodes.

Connect 4: Algorithms vs Default Opponent

Algorithm	Win %	Draw %	Loss %	Avg Time	Games
Minimax(d=4)	100.0%	0.0%	0.0%	2501.5ms	20
AlphaBeta(d=4)	100.0%	0.0%	0.0%	1339.0ms	20
Q-Learning	5.0%	0.0%	95.0%	0.2ms	20

DQN	10.0%	0.0%	90.0%	0.2ms	20

Table 3: Connect 4 performance against the default opponent (20 games each).

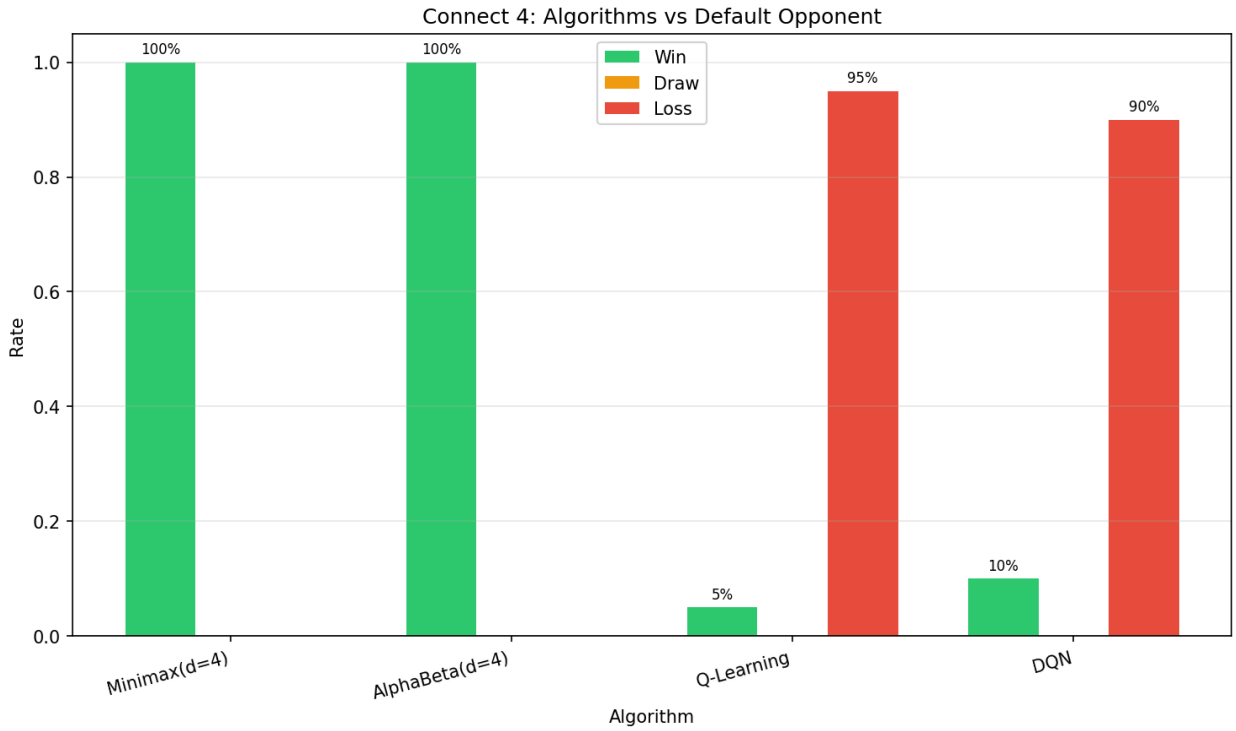

Figure 4: Win, draw, and loss rates for each algorithm against the default opponent in Connect 4.

Analysis

The performance gap between search based and learning based methods is dramatically amplified in Connect 4. Both Minimax variants achieved a perfect 100% win rate against the default opponent, even with only depth 4 lookahead. Alpha Beta pruning provides a clear computational advantage 1339.0ms per game versus Minimax's 2501.5ms, a 1.87x speedup.

Both RL agents performed poorly Q Learning won only 5% and DQN 10%. This is expected for several reasons. First, both were trained against a random opponent (as specified in the brief for Connect 4), so they learned strategies effective against random play that do not transfer to the default opponent's win/block heuristic. Second, Connect 4's state space is vastly larger than Tic Tac Toe's. Q Learning's table grew to 1.38 million entries in 30,000 episodes but this covers only a fraction of the approximately 4.5 trillion possible states.

Training Convergence

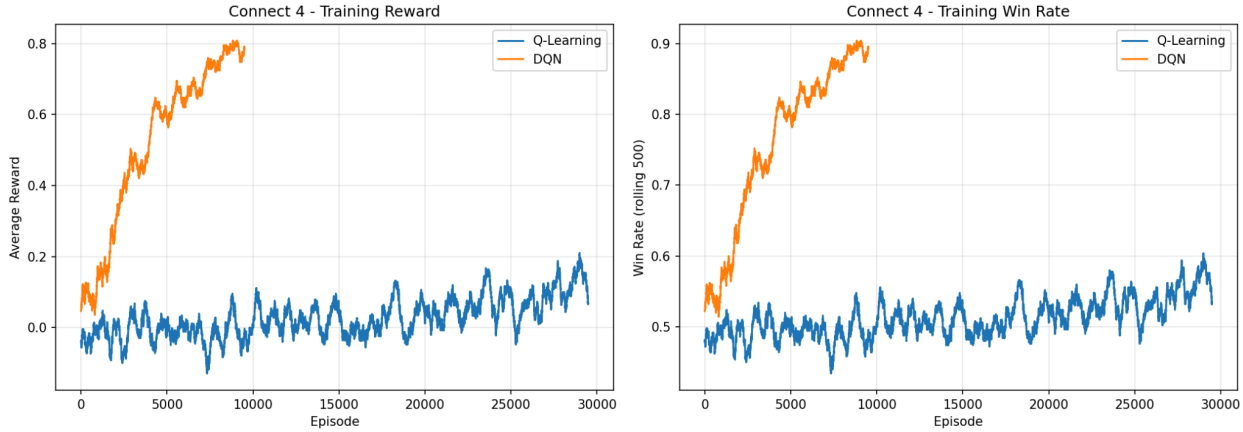

Figure 5: Training reward and win rate curves for RL agents on Connect 4 (rolling 500-episode average).

DQN shows strong learning against the random opponent, with its win rate climbing from 50% to nearly 89% over 10,000 episodes, demonstrating effective generalisation through function approximation. Q Learning struggles at around 50-55% win rate, barely above random, because its tabular approach cannot adequately cover the enormous state space with only 30,000 episodes. However, this superior training performance against random play does not translate to effectiveness against the more strategic default opponent.

Decision Time

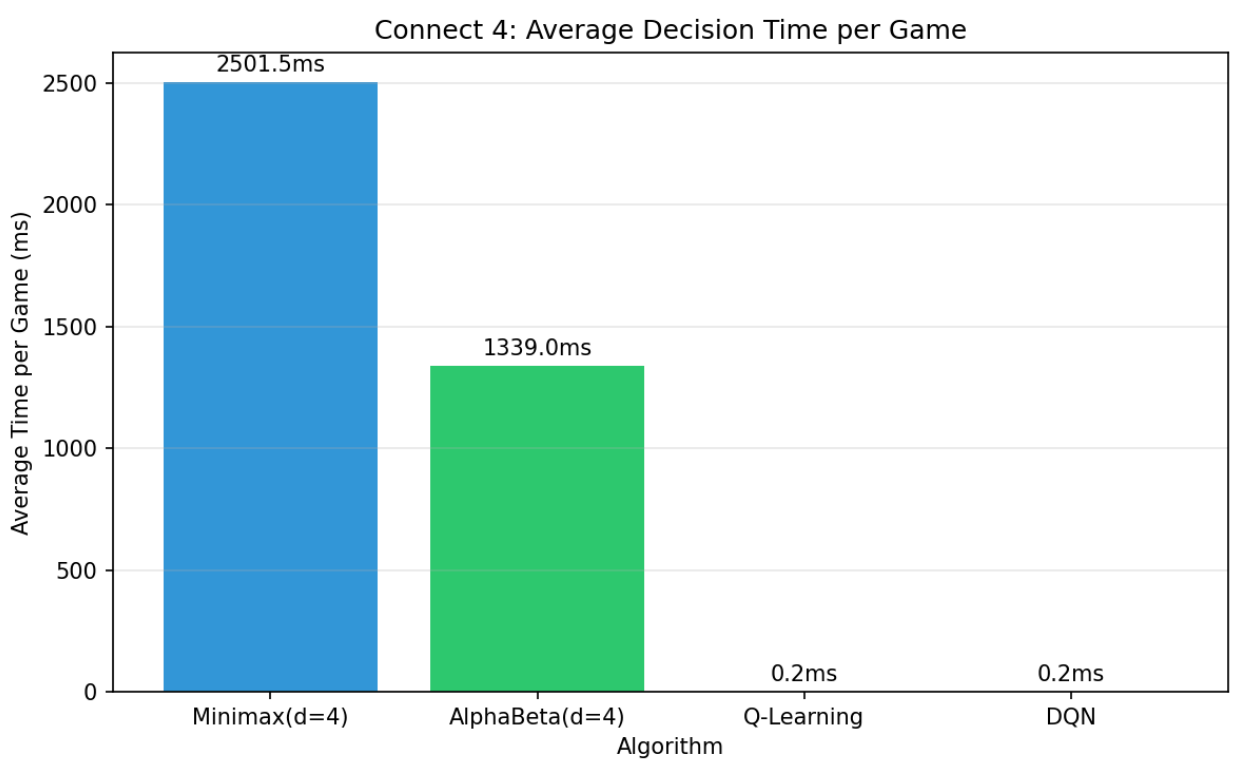

Figure 6: Average decision time per game for Connect 4.

The computational cost of search is much more apparent in Connect 4. Minimax(d=4) averages 2501.5ms per game and Alpha Beta(d=4) averages 1339.0ms, compared to 0.2ms for both RL agents. This represents a 6,500-12,500x speed difference, though the search agents deliver perfect win rates while the RL agents are largely ineffective.

Head to Head Matchups

Tic Tac Toe Head to Head

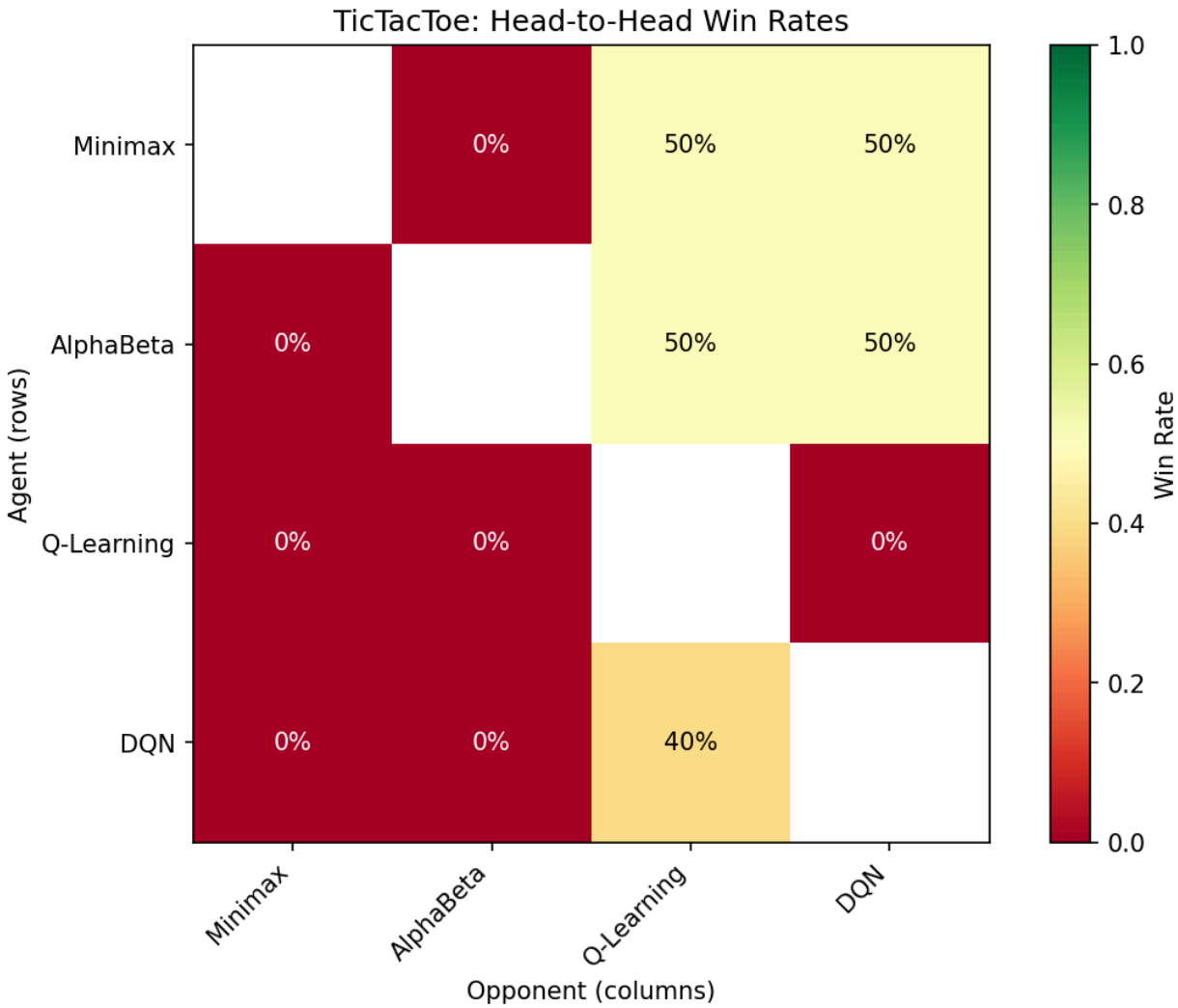

Figure 7: Head to head win rate heatmap for Tic Tac Toe (20 games per matchup).

Matchup	W	D	L	W%	D%	L%
Minimax vs AlphaBeta	0	20	0	0%	100%	0%

Minimax vs Q-Learning	10	10	0	50%	50%	0%
Minimax vs DQN	10	10	0	50%	50%	0%
AlphaBeta vs Q-Learning	10	10	0	50%	50%	0%
AlphaBeta vs DQN	10	10	0	50%	50%	0%
Q-Learning vs DQN	0	12	8	0%	60%	40%
DQN vs Q-Learning	8	12	0	40%	60%	0%

Table 4: Tic Tac Toe head to head results (20 games each).

The head to head results confirm a clear algorithmic hierarchy. Minimax and Alpha Beta are perfectly matched (100% draws), which is theoretically correct both play optimally and optimal TTT play from both sides always results in a draw. Neither RL agent ever beat a search based agent (0% win rate), while search agents won 50% of games (when going first) with the remainder being draws.

DQN outperformed Q Learning in their direct matchup, winning 40% with 60% draws while Q Learning won 0% against DQN. Despite DQN's lower win rate against the default opponent, its defensive play style proves more effective in head to head against Q Learning. This suggests DQN learned more robust positional play through function approximation, even if it failed to learn exploitative winning tactics.

Connect 4 Head to Head

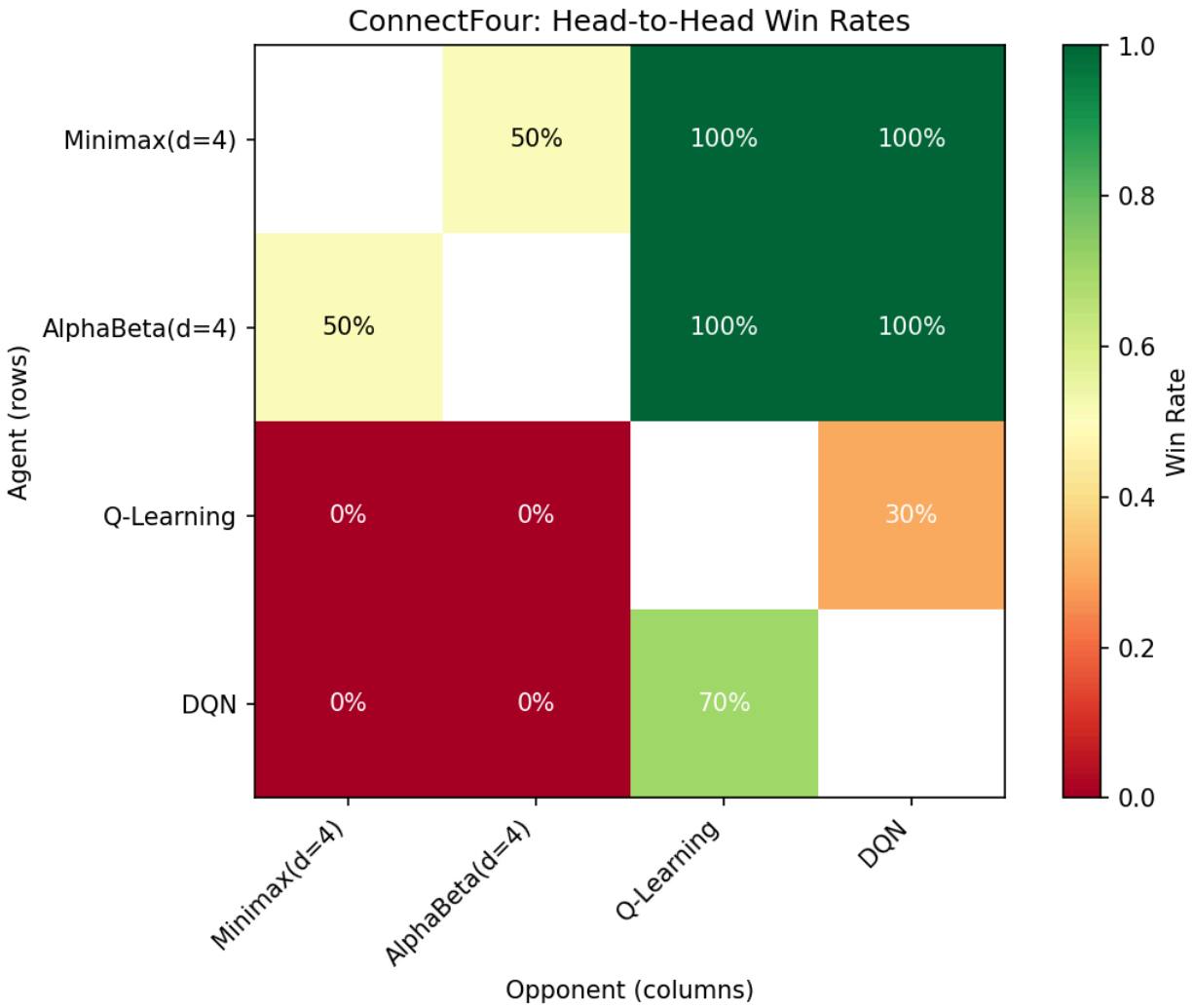

Figure 8: Head to head win rate heatmap for Connect 4 (20 games per matchup).

Matchup	W	D	L	W%	D%	L%
Minimax vs AlphaBeta	10	0	10	50%	0%	50%
Minimax vs Q-Learning	20	0	0	100%	0%	0%
Minimax vs DQN	20	0	0	100%	0%	0%

AlphaBeta vs Q-Learning	20	0	0	100%	0%	0%
AlphaBeta vs DQN	20	0	0	100%	0%	0%
Q-Learning vs DQN	6	0	14	30%	0%	70%
DQN vs Q-Learning	14	0	6	70%	0%	30%

Table 5: Connect 4 head to head results (20 games per matchup).

Both Minimax and Alpha Beta achieved 100% win rates against both RL agents, with no draws. Minimax vs Alpha Beta split exactly 50-50, determined entirely by first move advantage (the player who goes first wins every game when both use depth-4 search with the same evaluation function). This first player advantage is a known property of Connect 4.

DQN significantly outperformed Q Learning in their direct matchup, winning 70% versus Q Learning's 30%. This confirms that DQN's function approximation generalises better even in the larger Connect 4 state space. The zero draw rate across all Connect 4 matchups is notable unlike Tic Tac Toe where draws are common, Connect 4's deeper game tree means any imbalance leads to a decisive win.

Overall Comparison

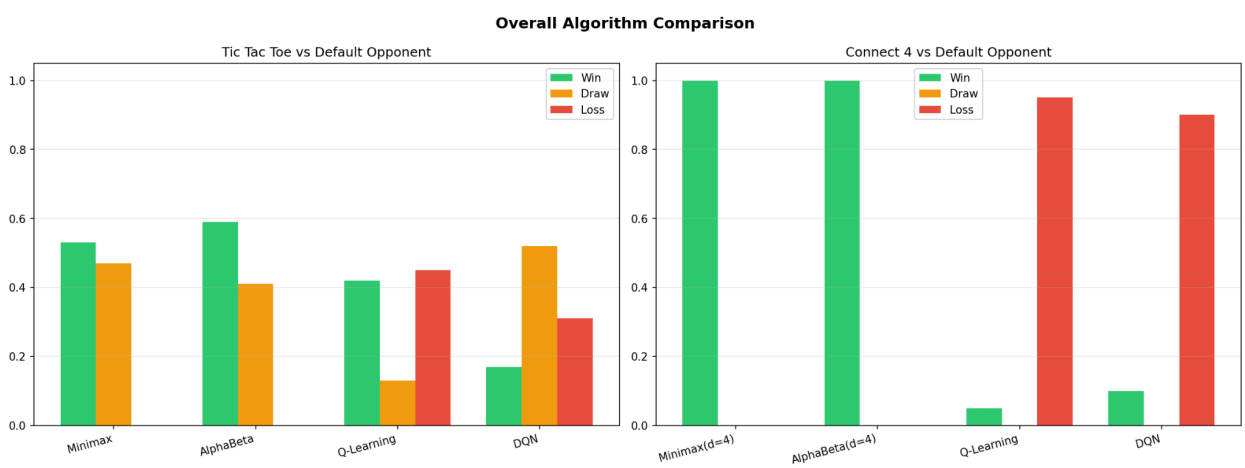

Figure 9: Overall comparison of algorithm performance across both games.

Game	Algorithm	Win%	Draw%	Loss%	Time(ms)

TTT	Minimax	53%	47%	0%	67.8
TTT	AlphaBeta	59%	41%	0%	126.4
TTT	Q-Learning	42%	13%	45%	0.0
TTT	DQN	17%	52%	31%	0.1
C4	Minimax(d=4)	100%	0%	0%	2501.5
C4	AlphaBeta(d=4)	100%	0%	0%	1339.0
C4	Q-Learning	5%	0%	95%	0.2
C4	DQN	10%	0%	90%	0.2

Table 6: Complete results summary across both games.

Playing strength: Search based methods are definitively superior for both games. They achieve 0% loss rates on Tic Tac Toe and 100% win rates on Connect 4. The RL agents achieve competitive but imperfect play on TTT and are largely ineffective on Connect 4.

Scalability: The performance gap widens dramatically as game complexity increases. In TTT (~5,478 states), Q Learning is competitive (42% wins). On Connect 4 (~4.5 trillion states), Q Learning's tabular approach collapses to 5% wins. DQN handles scaling slightly better (10% wins on C4) through function approximation but remains far from competitive with search.

Speed vs quality: RL agents are 600-12,500x faster at decision time (sub millisecond vs seconds), but this speed comes at the cost of playing strength. For applications where decision speed matters more than optimality, RL could be preferred.

Alpha Beta pruning value: Alpha Beta consistently reduces computation without sacrificing playing strength. On Connect 4, it provides a 1.87x speedup over plain Minimax. The scalability test showed a 3.66x reduction in nodes visited at depth 8.

Statistical Analysis

Tic Tac Toe Multi Run Analysis

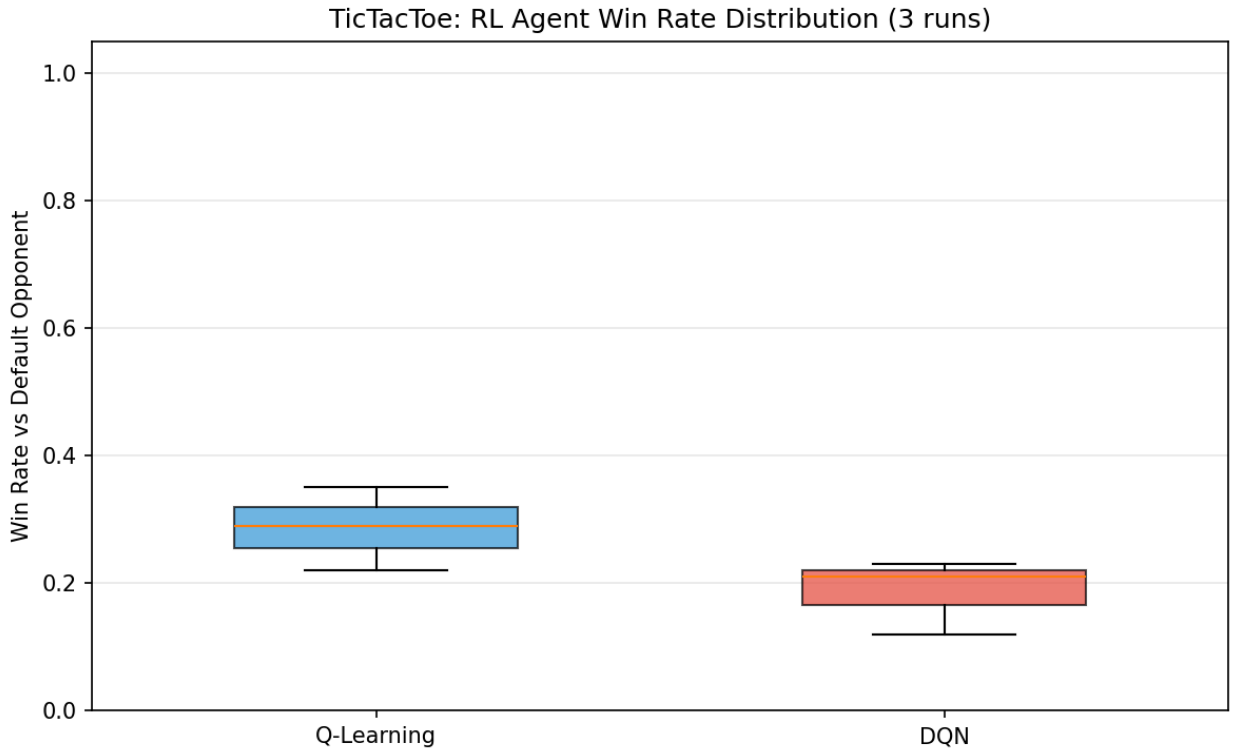

Figure 10: Win rate distribution of RL agents across 3 independent training runs on Tic Tac Toe.

Agent	Run 1	Run 2	Run 3	Mean	Std
Q-Learning	35%	29%	22%	28.7%	5.3%
DQN	12%	21%	23%	18.7%	4.8%

Table 7: Multi run statistical analysis of RL agents on Tic Tac Toe.

Q Learning showed moderate variance (std 5.3% around mean 28.7%), indicating reasonably stable training influenced by random exploration. DQN showed similar variance (4.8% around 18.7%). Interestingly, Q Learning's mean win rate in the multi-run analysis (28.7%) is lower than its single run result (42%), highlighting the stochastic nature of RL training and the importance of multi run evaluation.

Connect 4 Multi Run Analysis

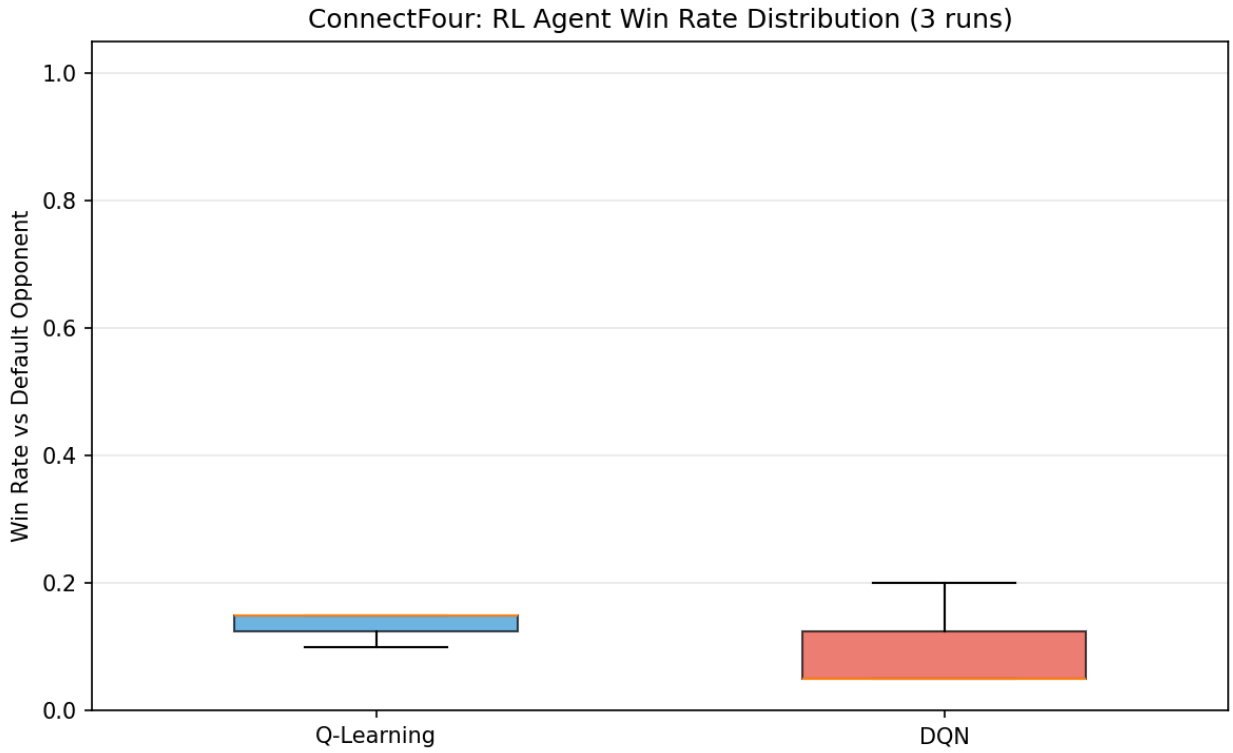

Figure 11: Win rate distribution of RL agents across 3 independent training runs on Connect 4.

Agent	Run 1	Run 2	Run 3	Mean	Std
Q-Learning	15%	15%	10%	13.3%	2.4%
DQN	20%	5%	5%	10.0%	7.1%

Table 8: Multi run statistical analysis of RL agents on Connect 4.

On Connect 4, Q Learning showed low variance (2.4%) but consistently poor performance (mean 13.3%). DQN showed high variance (7.1%) with one outlier run at 20% and two at 5%, indicating that DQN's function approximation is more sensitive to random initialisation in the larger state space. Neither agent consistently outperforms the other on Connect 4, unlike on TTT where Q Learning was consistently stronger.

Hyperparameter Sensitivity Analysis

Connect 4: Search Depth Variation

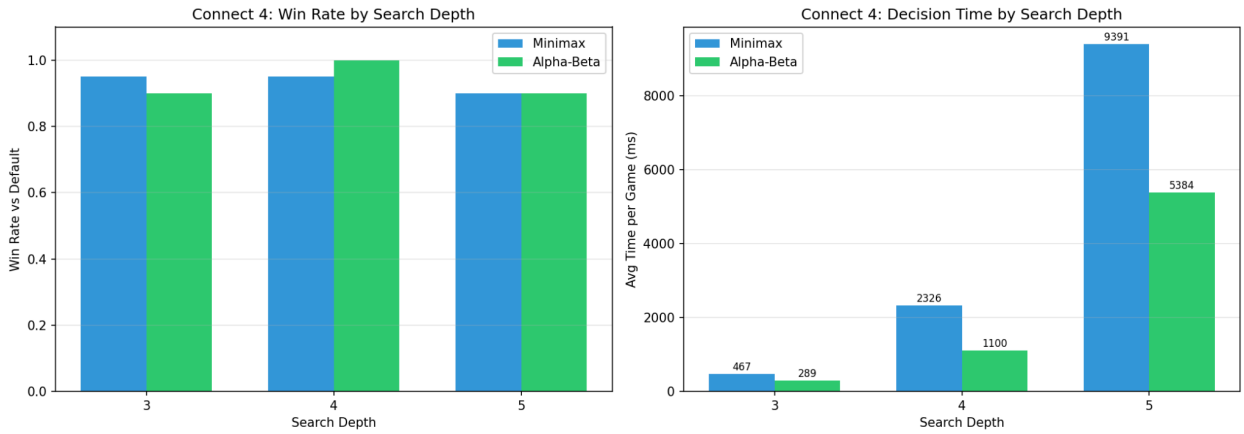

Figure 12: Connect 4 win rate and decision time as search depth varies from 3 to 5.

Depth	MM Win%	MM Time(ms)	AB Win%	AB Time(ms)
3	95.0%	467	90.0%	289
4	95.0%	2326	100.0%	1100
5	90.0%	9391	90.0%	5384

Table 9: Connect 4 depth variation results (20 games at d=3,4 10 games at d=5).

Increasing search depth does not monotonically improve win rate. Alpha Beta peaks at 100% at depth 4, while depth 3 and depth 5 both achieve 90%. This counterintuitive result at depth 5 occurs because the evaluation function's accuracy degrades when evaluating positions that are further from terminal states and the additional depth introduces noise from the heuristic.

Meanwhile, computational cost grows exponentially. Depth 5 Minimax takes 9391ms per game (20x slower than depth 3's 467ms). Alpha Beta's pruning advantage also grows with depth at depth 5, Alpha Beta is 1.74x faster than Minimax, compared to 1.62x at depth 3.

Depth 4 represents the best trade off between playing strength and computational cost, achieving the highest win rate (100% for Alpha Beta) at a reasonable 1100ms per game.

TTT: Q Learning Training Episode Variation

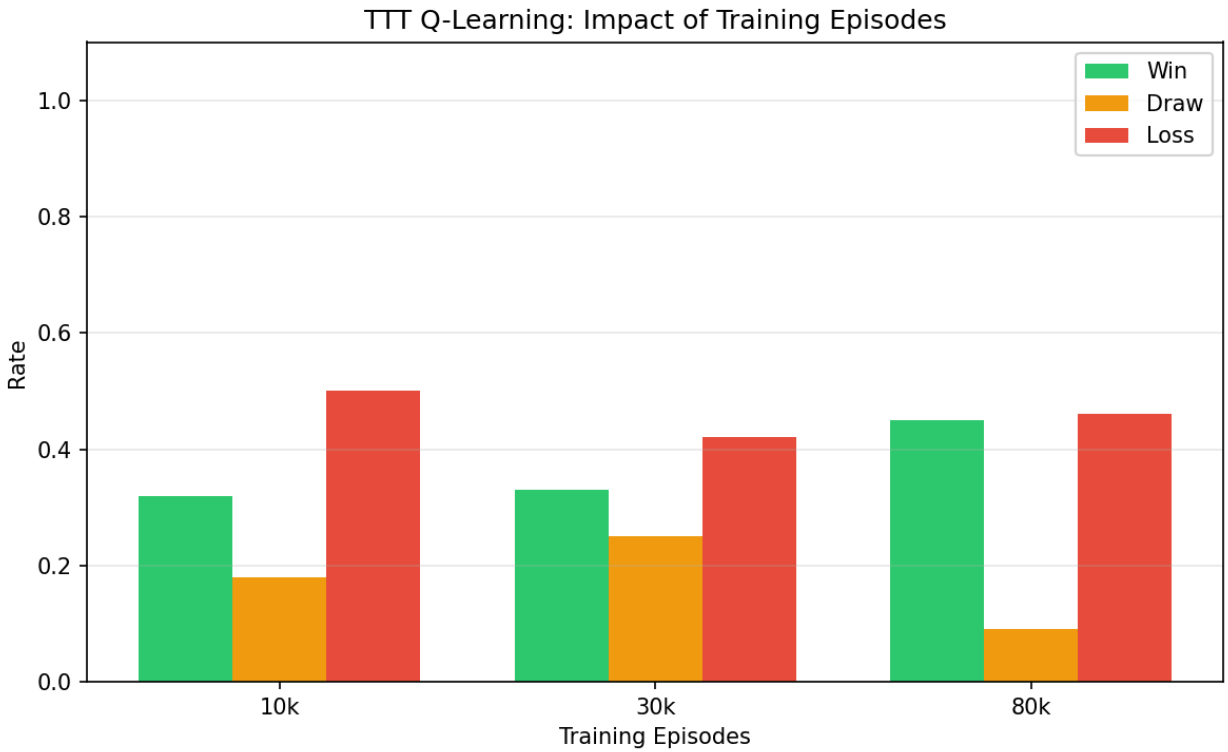

Figure 13: Impact of training episode count on Q Learning performance in Tic Tac Toe.

Episodes	Win%	Draw%	Loss%	Q Table Size
10,000	32.0%	18.0%	50.0%	7,670
30,000	33.0%	25.0%	42.0%	7,516
80,000	45.0%	9.0%	46.0%	7,982

Table 10: Q Learning performance at different training episode counts.

Win rate improves from 32% at 10,000 episodes to 45% at 80,000, demonstrating clear benefit from additional training. Interestingly, the Q table size remains relatively stable (~7,500-8,000 entries) across all three settings, indicating that the state space coverage saturates early but the quality of Q values continues to improve with more updates. Draw rate decreases from 18% to 9% as the agent becomes more decisive, while loss rate remains roughly constant, suggesting the additional training primarily converts draws into wins.

TTT: Q Learning Learning Rate Variation

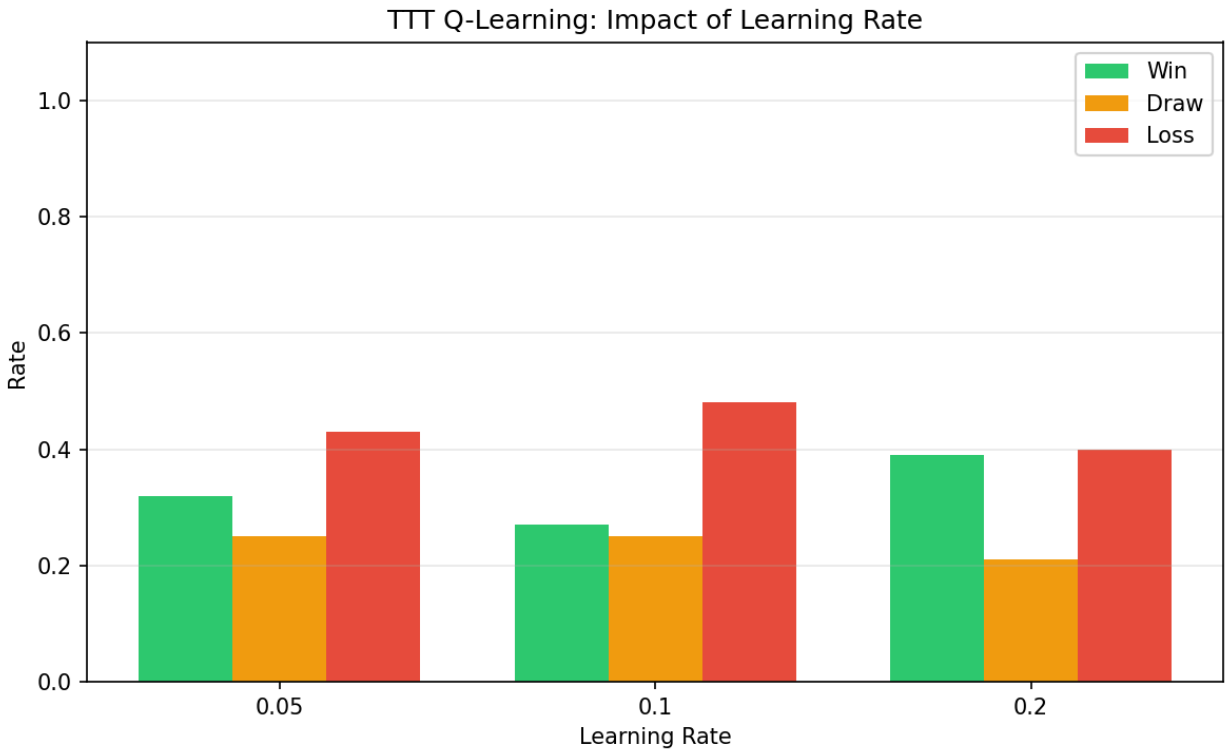

Figure 14: Impact of learning rate on Q Learning performance in Tic Tac Toe.

Learning Rate	Win%	Draw%	Loss%
0.05	32.0%	25.0%	43.0%
0.1	27.0%	25.0%	48.0%
0.2	39.0%	21.0%	40.0%

Table 11: Q Learning performance at different learning rates.

A learning rate of 0.2 produced the best win rate (39%), followed by 0.05 (32%) and 0.1 (27%). The higher learning rate allows the agent to adapt more quickly to the default opponent's strategy, at the potential cost of stability. The moderate learning rate of 0.1 performed worst, suggesting it is too slow to converge within the given episode budget but not stable enough to avoid oscillation. These results highlight the sensitivity of Q Learning to hyperparameter selection and the importance of tuning.

Limitations

Several limitations should be noted. The RL agents were trained against different opponents for TTT (default) versus C4 (random), making direct cross game comparison of RL performance less meaningful. The DQN implementation uses a simple feedforward network without convolutional layers, which limits its ability to detect spatial patterns in the Connect 4 board. The number of evaluation games (20 for C4, 100 for TTT) introduces sampling noise, particularly for C4 where results are more variable. The multi run analysis used only 3 runs, which limits the statistical power of variance estimates.

Conclusion

This analysis demonstrates that adversarial search algorithms (Minimax and Alpha Beta) are definitively superior to reinforcement learning methods (Q Learning and DQN) for deterministic two player zero sum games, particularly as game complexity increases.

First, search based methods provide optimal or near optimal play with no training required. Full depth Minimax is provably optimal on Tic Tac Toe, and even depth limited search on Connect 4 achieves 100% win rates against heuristic opponents.

Second, RL methods face fundamental scalability challenges. Tabular Q Learning is limited by the curse of dimensionality (1.38 million Q table entries for Connect 4 is still insufficient). DQN's function approximation partially addresses this, as evidenced by its superior training performance on Connect 4 (89% vs 53% win rate against random), but the learned representations do not transfer effectively to opponents using different strategies.

Third, there is a clear trade off between decision quality and decision speed. RL agents operate 3-4 orders of magnitude faster at decision time but produce significantly weaker play.

Fourth, hyperparameter sensitivity is significant for RL methods. Q Learning's performance varied from 27% to 45% win rate depending on learning rate and training episodes, while search based methods are parameter free (aside from depth limit, which is constrained by computational budget).

Future work could explore self play training for RL agents (as used in AlphaZero), deeper networks with convolutional architectures for Connect 4, Monte Carlo Tree Search as a hybrid approach or curriculum learning where RL agents train against progressively stronger opponents.

References

[1] Russell, S. and Norvig, P. (2021). Artificial Intelligence: A Modern Approach, 4th ed. Pearson.

[2] Sutton, R.S. and Barto, A.G. (2018). Reinforcement Learning: An Introduction, 2nd ed. MIT Press.

[3] Mnih, V., et al. (2015). Human level control through deep reinforcement learning. Nature, 518(7540), pp.529-533.

[4] Knuth, D.E. and Moore, R.W. (1975). An analysis of alpha beta pruning. Artificial Intelligence, 6(4), pp.293-326.

[5] Allis, V. (1988). A Knowledge based Approach of Connect Four. MSc Thesis, Vrije Universiteit.

[6] CS7IS2 Lecture Notes: Adversarial Search, Reinforcement Learning, and Markov Decision Processes. Trinity College Dublin, 2026.

Appendix 1: Minimax for Tic Tac Toe

File: minimax_algorithms.py, class MinimaxAgent

Configuration: max_depth=None (full depth search). On Tic Tac Toe's 3x3 board (max 9 moves), full depth search is feasible. A transposition table caches evaluated states to avoid redundant computation. The evaluation function (evaluate_tictactoe) is not used since search reaches terminal states.

```
def get_eval_function(game):

    if game.name == "TicTacToe":

        return evaluate_tictactoe

    else:

        return evaluate_connect4




class MinimaxAgent:

    # Standard minimax with transposition table. Supports depth limiting.



    def __init__(self, game, max_depth=None, player=1):

        self.game = game

        self.max_depth = max_depth

        self.player = player

        self.eval_fn = get_eval_function(game)

        self.nodes_visited = 0

        self.transposition_table = {}



    def select_action(self, state, player=None):

        if player is not None:

                self.player = player

        self.nodes_visited = 0
```

```
    self.transposition_table = {}

    valid_actions = self.game.get_valid_actions(state)
    best_action = valid_actions[0]
    best_value = -math.inf

    for action in valid_actions:
            next_state = self.game.make_move(state, action, self.player)
            value = self._min_value(next_state, depth=1)
            if value > best_value:
            best_value = value
            best_action = action

    return best_action

def _max_value(self, state, depth):
    self.nodes_visited += 1

    state_key = (self.game.state_to_tuple(state), True, depth)
    if state_key in self.transposition_table:
            return self.transposition_table[state_key]

    done, winner = self.game.check_winner(state)
    if done:
            if winner == self.player:
            val = 1000 - depth
            elif winner == -self.player:
            val = -1000 + depth
            else:
            val = 0
            self.transposition_table[state_key] = val
            return val
```

```
    if self.max_depth is not None and depth >= self.max_depth:
            val = self.eval_fn(state, self.game, self.player)
            self.transposition_table[state_key] = val
            return val

    value = -math.inf
    for action in self.game.get_valid_actions(state):
            next_state = self.game.make_move(state, action, self.player)
            value = max(value, self._min_value(next_state, depth + 1))

    self.transposition_table[state_key] = value
    return value

def _min_value(self, state, depth):
    self.nodes_visited += 1

    state_key = (self.game.state_to_tuple(state), False, depth)
    if state_key in self.transposition_table:
            return self.transposition_table[state_key]

    done, winner = self.game.check_winner(state)
    if done:
            if winner == self.player:
            val = 1000 - depth
            elif winner == -self.player:
            val = -1000 + depth
            else:
            val = 0
            self.transposition_table[state_key] = val
            return val

    if self.max_depth is not None and depth >= self.max_depth:
            val = self.eval_fn(state, self.game, self.player)
```

```
                self.transposition_table[state_key] = val
                return val
```

Evaluation function for Tic Tac Toe (used only if depth-limited):

```
def evaluate_tictactoe(state, game, player):
    # Heuristic: score based on how many lines each player threatens
    score = 0
    lines = []

    # Rows
    for r in range(3):
        lines.append([state[r][c] for c in range(3)])
    # Columns
    for c in range(3):
        lines.append([state[r][c] for r in range(3)])
    # Diagonals
    lines.append([state[i][i] for i in range(3)])
    lines.append([state[i][2 - i] for i in range(3)])

    for line in lines:
        player_count = line.count(player)
        opponent_count = line.count(-player)

        if opponent_count == 0:
                if player_count == 3:
                score += 100
                elif player_count == 2:
                score += 10
                elif player_count == 1:
                score += 1
        if player_count == 0:
```

```
                if opponent_count == 3:

                score -= 100

                elif opponent_count == 2:

                score -= 10

                elif opponent_count == 1:

                score -= 1


    return score
```

Appendix 2: Minimax for Connect 4

File: minimax_algorithms.py, class MinimaxAgent and class AlphaBetaAgent

Configuration: max_depth=4 (depth limited search). Full depth search on Connect 4's 6x7 board is infeasible (scalability test confirmed only depth 8/42 reachable in 90s). Alpha Beta pruning is applied to reduce nodes visited by approximately 3.66x. The evaluate_connect4 heuristic scores board positions based on connected piece windows across all four directions.

Alpha Beta Agent (used for Connect 4 with depth limit):

```
                next_state = self.game.make_move(state, action, -self.player)

                value = min(value, self._max_value(next_state, depth + 1))


        self.transposition_table[state_key] = value

        return value




class AlphaBetaAgent:

    # Minimax with alpha-beta pruning. Supports depth limiting.


    def __init__(self, game, max_depth=None, player=1):

        self.game = game

        self.max_depth = max_depth
```

```
    self.player = player
    self.eval_fn = get_eval_function(game)
    self.nodes_visited = 0

def select_action(self, state, player=None):
    if player is not None:
            self.player = player
    self.nodes_visited = 0

    valid_actions = self.game.get_valid_actions(state)
    best_action = valid_actions[0]
    best_value = -math.inf
    alpha = -math.inf
    beta = math.inf

    for action in valid_actions:
            next_state = self.game.make_move(state, action, self.player)
            value = self._min_value(next_state, depth=1, alpha=alpha, beta=beta)
            if value > best_value:
            best_value = value
            best_action = action
            alpha = max(alpha, best_value)

    return best_action

def _max_value(self, state, depth, alpha, beta):
    self.nodes_visited += 1

    done, winner = self.game.check_winner(state)
    if done:
            if winner == self.player:
            return 1000 - depth
            elif winner == -self.player:
```

```
            return -1000 + depth
            else:
            return 0

    if self.max_depth is not None and depth >= self.max_depth:
            return self.eval_fn(state, self.game, self.player)

    value = -math.inf
    for action in self.game.get_valid_actions(state):
            next_state = self.game.make_move(state, action, self.player)
            value = max(value, self._min_value(next_state, depth + 1, alpha, beta))
            if value >= beta:
            return value
            alpha = max(alpha, value)
    return value

def _min_value(self, state, depth, alpha, beta):
    self.nodes_visited += 1

    done, winner = self.game.check_winner(state)
    if done:
            if winner == self.player:
            return 1000 - depth
            elif winner == -self.player:
            return -1000 + depth
            else:
            return 0

    if self.max_depth is not None and depth >= self.max_depth:
            return self.eval_fn(state, self.game, self.player)
```

Evaluation function for Connect 4:

```
rows, cols, win_len = game.rows, game.cols, game.win_length

opponent = -player

score = 0


def score_window(window):

    s = 0

    p_count = np.count_nonzero(window == player)

    o_count = np.count_nonzero(window == opponent)

    empty = np.count_nonzero(window == 0)


    if p_count == win_len:

            s += 1000

    elif p_count == win_len - 1 and empty == 1:

            s += 50

    elif p_count == win_len - 2 and empty == 2:

            s += 5


    if o_count == win_len:

            s -= 1000

    elif o_count == win_len - 1 and empty == 1:

            s -= 50


    return s


centre_col = cols // 2

centre_array = state[:, centre_col]

centre_count = np.count_nonzero(centre_array == player)

score += centre_count * 3


for r in range(rows):

    for c in range(cols - win_len + 1):

            window = state[r, c:c + win_len]

            score += score_window(window)
```

```
    for r in range(rows - win_len + 1):
        for c in range(cols):
                window = state[r:r + win_len, c]
                score += score_window(window)

    for r in range(rows - win_len + 1):
        for c in range(cols - win_len + 1):
                window = np.array([state[r + i][c + i] for i in range(win_len)])
                score += score_window(window)

    for r in range(win_len - 1, rows):
        for c in range(cols - win_len + 1):
                window = np.array([state[r - i][c + i] for i in range(win_len)])
                score += score_window(window)

    return score
```

Appendix 3: Q Learning for Tic Tac Toe

File: rl_algorithms.py, class QLearningAgent

Configuration: 80,000 training episodes against the default opponent. Learning rate=0.1, discount factor=0.95, epsilon decay from 1.0 to 0.01. The Q table maps (state_tuple, action) pairs to learned values. Final Q table size: approximately 7,627 entries covering the reachable TTT state space.

```
class QLearningAgent:
    # Tabular Q-Learning with epsilon-greedy exploration.

    def __init__(self, game, player=1, learning_rate=0.1, discount_factor=0.95,
                epsilon_start=1.0, epsilon_end=0.01, epsilon_decay=0.9995):
        self.game = game
```

```
    self.player = player
    self.lr = learning_rate
    self.gamma = discount_factor
    self.epsilon = epsilon_start
    self.epsilon_start = epsilon_start
    self.epsilon_end = epsilon_end
    self.epsilon_decay = epsilon_decay

    self.q_table = defaultdict(float)

    self.training_rewards = []
    self.training_wins = []
    self.training_epsilon = []

def _get_q_value(self, state, action):
    state_key = self.game.state_to_tuple(state)
    return self.q_table[(state_key, action)]

def _get_max_q(self, state):
    valid_actions = self.game.get_valid_actions(state)
    if not valid_actions:
            return 0.0
    return max(self._get_q_value(state, a) for a in valid_actions)

def _update_q(self, state, action, reward, next_state, done):
    state_key = self.game.state_to_tuple(state)
    old_q = self.q_table[(state_key, action)]

    if done:
            target = reward
    else:
            target = reward + self.gamma * self._get_max_q(next_state)
```

```
    self.q_table[(state_key, action)] = old_q + self.lr * (target - old_q)


def select_action(self, state, player=None, training=False):
    if player is not None:
            self.player = player


    valid_actions = self.game.get_valid_actions(state)
    if not valid_actions:
            return None


    if training and random.random() < self.epsilon:
            return random.choice(valid_actions)


    best_value = -math.inf
    best_actions = []
    for action in valid_actions:
            q_val = self._get_q_value(state, action)
            if q_val > best_value:
            best_value = q_val
            best_actions = [action]
            elif q_val == best_value:
            best_actions.append(action)


    return random.choice(best_actions)


def train(self, opponent, num_episodes=50000, verbose=True):
    self.epsilon = self.epsilon_start
    win_count = 0
    window_size = 500


    for episode in range(num_episodes):
            state = self.game.reset()
            done = False
```

```
episode_reward = 0

winner = 0


agent_turn = random.choice([True, False])


if not agent_turn:

opp_action = opponent.select_action(state, -self.player)

state = self.game.make_move(state, opp_action, -self.player)

done, winner = self.game.check_winner(state)

agent_turn = True


while not done:

action = self.select_action(state, training=True)

next_state = self.game.make_move(state, action, self.player)

done, winner = self.game.check_winner(next_state)


if done:

        if winner == self.player:

        reward = 1.0

        win_count += 1

        elif winner == -self.player:

        reward = -1.0

        else:

        reward = 0.3

        self._update_q(state, action, reward, next_state, done)

        episode_reward += reward

        break


# Opponent's turn

opp_action = opponent.select_action(next_state, -self.player)

after_opp_state = self.game.make_move(next_state, opp_action, -self.player)

done, winner = self.game.check_winner(after_opp_state)
```

```
        if done:
                if winner == self.player:
                reward = 1.0
                win_count += 1
                elif winner == -self.player:
                reward = -1.0
                else:
                reward = 0.3
                self._update_q(state, action, reward, after_opp_state, done)
                episode_reward += reward
                break

        self._update_q(state, action, 0.0, after_opp_state, False)
        state = after_opp_state

        self.epsilon = max(self.epsilon_end, self.epsilon * self.epsilon_decay)

    self.training_rewards.append(episode_reward)
        self.training_wins.append(1 if (done and winner == self.player) else 0)
    self.training_epsilon.append(self.epsilon)

        if verbose and (episode + 1) % (num_episodes // 10) == 0:
        recent_wins = sum(self.training_wins[-window_size:])
        recent_total = min(window_size, episode + 1)
        win_rate = recent_wins / recent_total
        print(f"  Episode {episode + 1}/{num_episodes} | "
                f"Win rate (last {recent_total}): {win_rate:.2%} | "
                f"Epsilon: {self.epsilon:.4f} | "
                f"Q-table size: {len(self.q_table):,}")

if verbose:
        total_wins = sum(self.training_wins)
        print(f"  Training complete. Total wins: {total_wins}/{num_episodes} "
```

```
            f"({total_wins / num_episodes:.2%})")


        return self.training_rewards, self.training_wins
```

Appendix 4: Q Learning for Connect 4

File: rl_algorithms.py, class QLearningAgent (same class as Appendix 3)

Configuration: 30,000 training episodes against a random opponent (as specified in the assignment brief for the larger Connect 4 state space). Epsilon decay=0.99995 (slower than TTT due to larger exploration needs). Final Q table size: approximately 1,383,307 entries, covering only a small fraction of the estimated 4.5 trillion possible Connect 4 states.

The same QLearningAgent class is used; the game interface abstraction allows it to operate on either game without modification. The key difference is the training opponent (random vs default) and hyperparameter tuning for the larger state space.

[Same code as Appendix 3 - QLearningAgent class handles both games via the game interface]

Appendix 5: DQN for Tic Tac Toe

File: rl_algorithms.py, class DQNAgent

Configuration: Network architecture Input(9) -> Hidden(64, ReLU) -> Hidden(32, ReLU) -> Output(9). Trained for 10,000 episodes against the default opponent. Learning rate=0.001, batch size=32, replay buffer=10,000, target network soft update tau=0.1 every 10 episodes. Features: Huber loss for gradient stability, Adam optimiser, experience replay, soft target network updates. Implemented entirely in NumPy with no external deep learning frameworks.

```
class DQNAgent:

    # DQN with numpy neural net, experience replay, and target network.

    # Architecture: Input -> 128 (ReLU) -> 64 (ReLU) -> Output


    def __init__(self, game, player=1, learning_rate=0.001, discount_factor=0.95,

                epsilon_start=1.0, epsilon_end=0.01, epsilon_decay=0.9995,

                batch_size=64, buffer_size=50000, target_update_freq=50,

                tau=0.01):

        self.game = game
```

```
self.player = player

self.lr = learning_rate

self.gamma = discount_factor

self.epsilon = epsilon_start

self.epsilon_start = epsilon_start

self.epsilon_end = epsilon_end

self.epsilon_decay = epsilon_decay

self.batch_size = batch_size

self.target_update_freq = target_update_freq

self.tau = tau


self.input_size = game.rows * game.cols

self.hidden1_size = 128

self.hidden2_size = 64

if game.name == "TicTacToe":

        self.num_actions = game.rows * game.cols  # 9

else:

        self.num_actions = game.cols  # 7


self.weights = self._init_weights()

self.target_weights = self._copy_weights(self.weights)


self.adam_m = {k: np.zeros_like(v) for k, v in self.weights.items()}

self.adam_v = {k: np.zeros_like(v) for k, v in self.weights.items()}

self.adam_t = 0

self.adam_beta1 = 0.9

self.adam_beta2 = 0.999

self.adam_eps = 1e-8


self.replay_buffer = deque(maxlen=buffer_size)


self.training_rewards = []

self.training_wins = []
```

```
        self.training_epsilon = []

        self.training_losses = []



    def _init_weights(self):

        # Xavier init

        np.random.seed(None)

        weights = {

                'W1': np.random.randn(self.input_size, self.hidden1_size) * np.sqrt(2.0 /
(self.input_size + self.hidden1_size)),

                'b1': np.zeros(self.hidden1_size),

                'W2': np.random.randn(self.hidden1_size, self.hidden2_size) * np.sqrt(2.0 /
(self.hidden1_size + self.hidden2_size)),

                'b2': np.zeros(self.hidden2_size),

                'W3': np.random.randn(self.hidden2_size, self.num_actions) * np.sqrt(2.0 /
(self.hidden2_size + self.num_actions)),

                'b3': np.zeros(self.num_actions),

        }

        return weights



    def _copy_weights(self, weights):

        return {k: v.copy() for k, v in weights.items()}



    def _soft_update_target(self):

        for key in self.weights:

                self.target_weights[key] = (self.tau * self.weights[key] +

                                            (1 - self.tau) * self.target_weights[key])



    def _forward(self, x, weights):

        z1 = x @ weights['W1'] + weights['b1']

        a1 = np.maximum(0, z1)



        z2 = a1 @ weights['W2'] + weights['b2']

        a2 = np.maximum(0, z2)



        q_values = a2 @ weights['W3'] + weights['b3']
```

```
    cache = {'x': x, 'z1': z1, 'a1': a1, 'z2': z2, 'a2': a2}
    return q_values, cache

def _backward_huber(self, q_values, targets, cache, actions):
    batch_size = q_values.shape[0]

    dq = np.zeros_like(q_values)
    for i in range(batch_size):
        diff = q_values[i, actions[i]] - targets[i]
        dq[i, actions[i]] = np.clip(diff, -1.0, 1.0) / batch_size

    dW3 = cache['a2'].T @ dq
    db3 = np.sum(dq, axis=0)
    da2 = dq @ self.weights['W3'].T
    da2 = da2 * (cache['z2'] > 0)

    dW2 = cache['a1'].T @ da2
    db2 = np.sum(da2, axis=0)
    da1 = da2 @ self.weights['W2'].T
    da1 = da1 * (cache['z1'] > 0)

    dW1 = cache['x'].T @ da1
    db1 = np.sum(da1, axis=0)

    grads = {'W1': dW1, 'b1': db1, 'W2': dW2, 'b2': db2, 'W3': dW3, 'b3': db3}

    # Gradient norm clipping
    total_norm = 0
    for key in grads:
        total_norm += np.sum(grads[key] ** 2)
    total_norm = np.sqrt(total_norm)
    max_norm = 1.0
```

```
        if total_norm > max_norm:
                scale = max_norm / (total_norm + 1e-8)
                for key in grads:
                grads[key] *= scale

        return grads

    def _adam_update(self, grads):
        self.adam_t += 1
        for key in self.weights:
                self.adam_m[key] = self.adam_beta1 * self.adam_m[key] + (1 - self.adam_beta1) *
grads[key]
                self.adam_v[key] = self.adam_beta2 * self.adam_v[key] + (1 - self.adam_beta2) *
(grads[key] ** 2)

                m_hat = self.adam_m[key] / (1 - self.adam_beta1 ** self.adam_t)
                v_hat = self.adam_v[key] / (1 - self.adam_beta2 ** self.adam_t)

                self.weights[key] -= self.lr * m_hat / (np.sqrt(v_hat) + self.adam_eps)

    def _state_to_input(self, state):
        return state.flatten().astype(np.float32)

    def _get_q_values(self, state, use_target=False):
        x = self._state_to_input(state).reshape(1, -1)
        weights = self.target_weights if use_target else self.weights
        q_values, _ = self._forward(x, weights)
        return q_values[0]

    def select_action(self, state, player=None, training=False):
        if player is not None:
                self.player = player

        valid_actions = self.game.get_valid_actions(state)
```

```
    if not valid_actions:
            return None

    if training and random.random() < self.epsilon:
            return random.choice(valid_actions)

    q_values = self._get_q_values(state)

    best_action = valid_actions[0]
    best_value = -math.inf
    for action in valid_actions:
            if q_values[action] > best_value:
            best_value = q_values[action]
            best_action = action

    return best_action

def _store_experience(self, state, action, reward, next_state, done):
    self.replay_buffer.append((
            self._state_to_input(state),
            action,
            reward,
            self._state_to_input(next_state),
            done
    ))

def _replay(self):
    if len(self.replay_buffer) < self.batch_size:
            return 0.0

    batch = random.sample(self.replay_buffer, self.batch_size)
    states, actions, rewards, next_states, dones = zip(*batch)
```

```
    states = np.array(states)
    actions = np.array(actions, dtype=int)
    rewards = np.array(rewards, dtype=np.float32)
    next_states = np.array(next_states)
    dones = np.array(dones, dtype=np.float32)

    rewards = np.clip(rewards, -1.0, 1.0)

    q_values, cache = self._forward(states, self.weights)
    next_q_values, _ = self._forward(next_states, self.target_weights)
    max_next_q = np.max(next_q_values, axis=1)

    targets = rewards + (1 - dones) * self.gamma * max_next_q
    targets = np.clip(targets, -2.0, 2.0)

    grads = self._backward_huber(q_values, targets, cache, actions)
    self._adam_update(grads)

    loss = 0.0
    for i in range(self.batch_size):
            diff = abs(q_values[i, actions[i]] - targets[i])
            if diff < 1.0:
            loss += 0.5 * diff * diff
            else:
            loss += diff - 0.5
    loss /= self.batch_size

    return loss

def train(self, opponent, num_episodes=50000, verbose=True):
    self.epsilon = self.epsilon_start
    win_count = 0
    window_size = 500
```

```
for episode in range(num_episodes):
        state = self.game.reset()
        done = False
        episode_reward = 0
        episode_loss = 0
        steps = 0
        winner = 0

        agent_turn = random.choice([True, False])

        if not agent_turn:
        opp_action = opponent.select_action(state, -self.player)
        state = self.game.make_move(state, opp_action, -self.player)
        done, winner = self.game.check_winner(state)
        agent_turn = True

        while not done:
        action = self.select_action(state, training=True)
        next_state = self.game.make_move(state, action, self.player)
        done, winner = self.game.check_winner(next_state)

        if done:
                if winner == self.player:
                reward = 1.0
                win_count += 1
                elif winner == -self.player:
                reward = -1.0
                else:
                reward = 0.3
            self._store_experience(state, action, reward, next_state, done)
                episode_reward += reward
                break
```

```
    # Opponent's turn
    opp_action = opponent.select_action(next_state, -self.player)
    after_opp_state = self.game.make_move(next_state, opp_action, -self.player)
    done, winner = self.game.check_winner(after_opp_state)

    if done:
            if winner == self.player:
            reward = 1.0
            win_count += 1
            elif winner == -self.player:
            reward = -1.0
            else:
            reward = 0.3
        self._store_experience(state, action, reward, after_opp_state, done)
            episode_reward += reward
            break

    self._store_experience(state, action, 0.0, after_opp_state, False)
    state = after_opp_state
    steps += 1

    if len(self.replay_buffer) >= self.batch_size:
    loss = self._replay()
    episode_loss = loss

    self.epsilon = max(self.epsilon_end, self.epsilon * self.epsilon_decay)

    if (episode + 1) % self.target_update_freq == 0:
    self._soft_update_target()

self.training_rewards.append(episode_reward)
    self.training_wins.append(1 if (done and winner == self.player) else 0)
```

```
    self.training_epsilon.append(self.epsilon)

    self.training_losses.append(episode_loss)


        if verbose and (episode + 1) % (num_episodes // 10) == 0:

        recent_wins = sum(self.training_wins[-window_size:])

        recent_total = min(window_size, episode + 1)

        win_rate = recent_wins / recent_total

        avg_loss = np.mean(self.training_losses[-window_size:])

        print(f"  Episode {episode + 1}/{num_episodes} | "

                f"Win rate (last {recent_total}): {win_rate:.2%} | "

                f"Epsilon: {self.epsilon:.4f} | "

                f"Avg loss: {avg_loss:.4f}")


if verbose:

        total_wins = sum(self.training_wins)

        print(f"  Training complete. Total wins: {total_wins}/{num_episodes} "

        f"({total_wins / num_episodes:.2%})")


return self.training_rewards, self.training_wins
```

Appendix 6: DQN for Connect 4

File: rl_algorithms.py, class DQNAgent (same class as Appendix 5)

Configuration: Network architecture Input(42) -> Hidden(128, ReLU) -> Hidden(64, ReLU) -> Output(7). Trained for 10,000 episodes against a random opponent. Learning rate=0.001, batch size=64, replay buffer=50,000, target network soft update tau=0.05 every 20 episodes. The larger network (128/64 vs 64/32) accommodates the higher dimensional input. Achieved 89% win rate against random during training but only 10% against the default opponent due to strategy mismatch.

The same DQNAgent class is used; the network dimensions are automatically configured based on the game's board size and action space. The key differences from TTT are the larger network, more replay buffer capacity, and training against a random rather than default opponent.

[Same code as Appendix 5 - DQNAgent class handles both games, with network size auto configured from game dimensions]